

A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time

Xinquan Wu^a, Xuefeng Yan^{a,b,*}, Donghai Guan^{a,b}, Mingqiang Wei^{a,b}

^a College of Computer Science and Technology, Nanjing University of Aeronautics and astronautics, Nanjing 210016, China

^b Collaborative Innovation Center of Novel Software Technology and Industrialization, Nanjing 210016, China

ARTICLE INFO

Dataset link: [github](https://github.com)

Keywords:

Dynamic job shop scheduling problem
Deep reinforcement learning
Prioritized experience replay
PPO
Uncertainty

ABSTRACT

The dynamic job-shop scheduling problem (DJSP) is a type of scheduling tasks where rescheduling is performed when encountering the uncertainties such as the uncertain operation processing time. However, the current deep reinforcement learning (DRL) scheduling approaches are hard to train convergent scheduling policies as the problem scale increases, which is very important for rescheduling under uncertainty. In this paper, we propose a DRL scheduling method for DJSP based on the proximal policy optimization (PPO) with hybrid prioritized experience replay. The job shop scheduling problem is formulated as a sequential decision-making problem based on Markov Decision Process (MDP) where a novel state representation is designed based on the feasible solution matrix which depicts the scheduling order of a scheduling task, a set of paired priority dispatching rules (PDR) are used as the actions and a new intuitive reward function is established based on the machine idle time. Moreover, a new hybrid prioritized experience replay method for PPO is proposed to reduce the training time where samples with positive temporal-difference (TD) error are replayed. Static experiments on classic benchmark instances show that the make-span obtained by our scheduling agent has been reduced by 1.59% on average than the best known DRL methods. In addition, dynamic experiments demonstrate that the training time of the reused scheduling policy is reduced by 27% compared with the retrained policy when encountering uncertainties such as uncertain operation processing time.

1. Introduction

The job shop scheduling problem (JSSP) is one of the most typical and important combinatorial optimization problems (COP) in the field of operational research and management science (Kayhan and Yildiz, 2021). It has been proven to be Nondeterministic Polynomial-time (NP) hard (Garey et al., 1976). The JSSP is a kind of typical machine scheduling problem which determines to assign a set of jobs on a set of machines to minimize or maximize a predefined objective function under certain constraints (Graham et al., 1979). JSSP has been concerned and studied extensively by scholars in engineering and academic fields and is widely applied in semiconductor (Park et al., 2020), machine manufacturing (Li et al., 2020), automobile manufacturing (Liu and Kozan, 2017), supply chain (Mohammadi et al., 2020), etc.

At present, the JSSP assumes that all the information of the manufacturing environment has been known in advance and there is no modification during the scheduling process. However, in the real-world industrial manufactures, the scheduling process is significantly affected by uncertainties (such as new job arrivals, machine breakdowns and uncertain processing time of operations, etc.). The broad application of

new information technologies such as Artificial Intelligence (AI), Cloud Computing, Big Data and Internet Of Things (IOT) in the manufacturing industry and the continuous development of integrated manufacturing innovations have been the main driving forces of the new industrial revolution (Zhou et al., 2018; Shaheen and Németh, 2022). In order to meet the requirements of the fourth industrial revolution, manufacturing industries are currently attempting to integrate these technologies into the Cyber-Physical Systems (CPS) for real-world manufacturing activities (Shaheen et al., 2023) where more and more uncertainties need to be considered. The dynamic job-shop scheduling problem (DJSP) which considers more uncertainties in manufacturing systems, has attracted an increasing interest of both academic researchers and practitioners.

In the past decades, numerous scheduling methods are proposed, which are mainly divided into mathematical programming, heuristic, meta-heuristic, hyper-heuristic and reinforcement learning (RL) methods. Since mathematical programming approaches suffer from the curse of dimensionality, it is infeasible for solving large-scale scheduling problems to find exact scheduling solutions in real-time. Although

* Corresponding author at: College of Computer Science and Technology, Nanjing University of Aeronautics and astronautics, Nanjing 210016, China.
E-mail address: yxf@nuaa.edu.cn (X. Yan).

simple priority dispatching rules (PDRs) (Sels et al., 2012) such as the shortest processing time (SPT) and the most work remaining (MWKR) react fast in dynamic scheduling environments, the performance of PDR methods varies significantly from instance to instance. Meta-heuristic methods, such as Genetic Algorithm (GA) (Mattfeld and Bierwirth, 2004), have an advantage over PDRs in terms of solution quality and robustness, however, these are usually more difficult to implement and tune, and computationally too complex to be used in a real-time system (Shahzad and Mebarki, 2012). Swarm Intelligence (SI) methods such as particle swarm optimization (Lin et al., 2010), ant colony optimization (Huang and Liao, 2008) and artificial bee colony algorithm (Asadzadeh, 2016) are also widely used to solve various scheduling problems. However, it is difficult to apply these meta-heuristic methods to the dynamic scheduling problems, because these methods should be restarted when encountering uncertainties, which is time-consuming. Hyper-heuristic methods such as genetic programming-based hyper-heuristic (GPHH) (Nguyen et al., 2017), have become one of the most effective methods to solve DJSP (Branke et al., 2016). However, the GP-evolved rules are often very large and complex, and are hard to interpret.

RL is one of the most remarkable branches of machine learning, which is usually formulated as a decision-making problem based on Markov Decision Process (MDP) to find optimal policies. In recent years, the RL approaches have been applied to DJSP due to the stochastic nature of RL (Kayhan and Yildiz, 2021). In RL scheduling methods, the scheduling agent should learn the scheduling policies which are usually expressed as arrays or tables. This type of tabular RL methods are only suitable for small-scale scheduling problems because of the curse of dimensionality. However, scheduling tasks that are more complex and closer to the real-world situations usually have a higher dimensional state space and large action space, which limits the application of RL.

The achievements of deep learning in image recognition (Roy et al., 2022; Kim et al., 2021), natural language processing (NLP) (Ullah et al., 2021), object detection (Roy and Bhaduri, 2022) and other domains have shown that it has a strong fitting capability as well as powerful representation capability when processing high-dimensional data (Bengio et al., 2021). Since the successful application of deep Q-network (DQN) in game playing (Mnih et al., 2013), more and more deep learning techniques have been combined with RL. Deep reinforcement learning (DRL) which integrates the powerful representation capability of deep neural networks and the decision-making ability of reinforcement learning, has also made some important advances in applications of other field, such as robot control (Tai et al., 2017), recommendation system (Zheng et al., 2018), autonomous driving (Russo et al., 2021), and computer vision (CV) (Yun et al., 2017). Meanwhile, DRL has been used to solve COP (Bengio et al., 2021), such as traveling salesman problems (TSP) (Ouyang et al., 2021) and vehicle route planning problems (VRP) (Yu et al., 2019). Therefore, DRL shows great potential to solve scheduling tasks with large or continuous state space.

In DRL scheduling methods, various artificial neural networks (ANN) are combined with RL such as convolutional neural networks (CNN), multi-layer perception (MLP), graph neural networks (GNN), recurrent neural networks (RNN) and attention mechanism-based networks. Although the current DRL methods for solving large-scale DJSP have achieved great progress, the performance of these DRL scheduling methods is far from the optimal scheduling solutions and some are even worse than single PDR scheduling methods (Tassel et al., 2021). Besides, the training time of current DRL scheduling agents is too long and it is hard to obtain a convergent scheduling policy for the DRL method due to the uncertain nature of deep neural networks (Gawlikowski et al., 2021). However, in the real-world manufacturing, the scheduling agent attempts to get more and more accurate scheduling solutions as soon as possible.

In RL, the agent learns how to act based on the state of the environment to maximize the expected long-term reward (Wang et al., 2022).

The quality of learning samples is very important to the performance of RL, which depends on the state representation and the design of reward. Motivated by the principles, we focus on generating high-quality samples by capturing more accurate state features of the scheduling environment and giving appropriate reward. Besides, the experience replay is designed to satisfy the assumption of independent and identically distribution and replay useful experiences more often, which can make the agent learn faster and improve the performance (Schaul et al., 2016). Therefore, we attempt to explore appropriate experience replay method for the actor-critic proximal policy optimization (PPO) (Schulman et al., 2017) to make the scheduling agent obtain more accurate solutions using less training time.

In this paper, we propose a DRL scheduling model based on PPO to solve the DJSP. The contributions of this work are listed as follows. (1) We propose to model the job shop scheduling problem as a single agent RL problem where the agent need to learn a scheduling policy to choose which job to work on at each step. (2) A novel state representation is designed based on the feasible solution matrix which requires less static job shop information and a novel intuitive reward function is established based on the machine idle time which is proved to be negatively linearly dependent with the maximum make-span of a scheduling task. (3) To make the scheduling policy easier to converge, a hybrid prioritized experience replay method for PPO is proposed where samples with positive TD-error are replayed. (4) We compare our scheduling method with state-of-the-art DRL scheduling methods on standard benchmark instances and the average make-span is reduced by 1.59%. Dynamic experiments show that the training time of the reused scheduling policy is reduced by 27% than the retrained policy when encountering operation processing time uncertainty.

The rest of the paper is organized as follows. Section 2 gives a brief review of intelligent scheduling methods for DJSP. The specific DRL scheduling model for DJSP is constructed in Section 3. In Section 4, static and dynamic experiments on standard benchmarks with different sizes are performed and the results are compared and discussed, while conclusions and future work reside in Section 5.

2. Literature review

In the past decades, numerous scheduling approaches have been proposed in the literature to solve the JSSP, including mathematical programming, heuristic, meta-heuristic, hyper-heuristic and RL scheduling methods.

The mathematical programming optimization approaches such as integer linear programming (Manne, 1960) and the mixed integer programming (Morinaga et al., 2023) can find the optimal solutions for small-scale problems but are impractical for large-scale or dynamic scheduling problems because of the curse of dimensionality and the inability to be modified during real-time execution.

Since the DJSP cannot be solved efficiently with these exact optimization methods, approximate solution optimization methods which aim to find near-optimal solutions, have been applied to solve DJSP. Heuristic dispatching rules which can deliver acceptable but not necessarily optimal solutions in a short computation time, are widely used to solve the DJSP (Yahyaoui et al., 2011). The manual design of effective dispatching rules is a trial-and-error practice, which requires a great deal of time, code effort and domain knowledge (Branke et al., 2016). Simple priority dispatching rule (PDR) such as the shortest processing time (SPT), is easy to implement and has low time-complexity, but the performance of PDR scheduling methods is significantly affected by instance-related factors such as shop configuration, operating condition and objective functions. Another kind of heuristic method shifting bottleneck procedure (SBP) (Adams et al., 1988) solves the scheduling problem through bottleneck identification and local reoptimization procedures, which can provide better solutions than the PDR, but it requires longer calculation time and more complicated implementation.

In traditional meta-heuristic methods, such as Tabu search (Geiger et al., 1997), simulated annealing (Yim and Lee, 1999), and genetic algorithm (Mattfeld and Bierwirth, 2004), random search factors are introduced to enhance the exploration ability and higher quality solutions can be provided in reasonable computational time but fine-tuning the parameters of these methods can be a challenge. As an emerging meta-heuristic method, swarm intelligence algorithms such as particle swarm optimization (Lin et al., 2010), ant colony optimization (Huang and Liao, 2008) and artificial bee colony algorithm (Asadzadeh, 2016) are also widely used to solve various scheduling problems. However, it is difficult to apply these meta-heuristic methods to the dynamic scheduling problems, where the conditions of the problem continuously change, because they are designed to find the entire scheduling assignments for a given initial condition. Therefore, these methods should be applied again whenever the conditions of the scheduling problem change (Baykasoglu and Karaslan, 2017), which is impractical in real world environment due to the extensive required computation time.

Hyper-heuristic methods such as genetic programming-based hyper-heuristic (GPHH) (Nguyen et al., 2017) which aim to automatically design dispatching rules in the heuristic space rather than the solution space, have become one of the most effective methods to solve DJSP (Branke et al., 2016). However, the generated rules from GPHH are often very large and complex, and are hard to interpret. To reduce the length of generated dispatching rules, many feature selection methods are employed in GPHH (Shady et al., 2021, 2022; Zhang et al., 2021). However, the current feature selection methods usually adapt offline selection mechanisms to obtain a promising subset of terminals, which require additional simulation runs, making the GP algorithm too time-consuming and impractical.

As one of the most remarkable branches of machine learning, the RL approach is widely used in production and operations management problems, especially in decision problems in dynamic environments. RL agent and the simulated environment are provided in Aydin and ÖZtemel (2000) to solve the DJSP where Q-III method is used to select the most appropriate PDR in real time and the RL agent gives better results than the traditional SPT, COVERT, and CR rules. In Shahrabi et al. (2017), RL with a Q-factor algorithm is used to optimize the parameters of variable neighborhood search (VNS) which is employed to find the optimal scheduling solution of DJSP. The parameters of VNS and Q-factors are updated until the next uncertain event takes place. Results indicate that the performance of this method is significantly better than those of the common dispatching rules and general VNS. Qu et al. (2016) employ RL to learn optimal dispatching rules for a small-scale, multi-stage, multi-machine, and multi-product type DJSP. These RL methods are usually used indirectly and are mainly suitable for small-scale scheduling problems with small state space. However, practical scheduling problems are usually solved in a multi-step mode and various selection are available in each step so that they often have a large or continuous state space, which limits the application of RL.

To solve this problem, DRL which combines these capabilities of deep learning and the decision-making ability of RL, is being applied to solve the DJSP with continuous state space and excessive action space (Wang et al., 2022). In DRL methods, the scheduling problem is usually thought as a sequential decision-making problem based on MDP which consists of state, action, reward function and transition probabilities. The solution of a MDP includes a policy which maps states to actions and is learned by gradient-based (such as PPO) or value-based (such as DQN) methods. We compared different DRL scheduling methods from the aspects of state, action, reward function, policy and learning methods, shown in Table 1.

In DRL scheduling methods, the dispatching rule is also called scheduling policy, which is employed to obtain a scheduling action given the state of a scheduling environment at each decision step. When confront uncertainties, the scheduling agent use the trained policy to make decisions for new tasks in real time. The scheduling policy is usually represented by deep neural networks such as CNN, MLP, and

other deep neural networks, which maps the high-dimensional state of the manufacturing environment to low dimensional dispatching actions of JSSP. However, the last layer of these deep neural networks is usually a fully connected layer or a classifier (Softmax), which needs to have a fixed-size input by their definition. As a result, these DRL methods usually require instance-by-instance training to derive the schedule policy, which cannot apply the trained policy to a new JSSP with different sizes.

The deep neural networks of scheduling policy are different due to the differences on the objectives and scenarios. In order to apply the trained scheduling policy to new tasks with different sizes, graph neural network (GNN) and graph embedding (GE) are employed to generate size-agnostic scheduling policy. In these DRL methods, disjunctive graphs (DG) is used to represent the state of the scheduling environment, GNN is utilized to encode the nodes in the disjunctive graphs and GE is exploited to generate fixed dimensional embeddings for a fully connected layer. MLP is subsequently used to compute action scores based on graph embeddings and Softmax function is employed to generate the action probability distribution. Following the same idea of GE, the ScheduleNet (Park et al., 2021a) which is a multi-agent scheduling method, utilizes the Type-aware Graph Attention (TGA) to extract the embeddings of agent and tasks nodes. Currently, this model achieves the state-of-the-art (SOTA) performance on the generalization capacity for JSSP with various sizes. This kind of scheduling policy can be trained on randomly generated instances and makes decisions on unseen instances with different sizes, which means that it can learn scheduling knowledge from other instances and use this knowledge to make decisions regardless the size of new instances.

Although the scheduling policies in these DRL scheduling methods are size-agnostic, the performance of these scheduling agents is far from the optimal scheduling solutions and some are no better than single PDR scheduling methods (Tassel et al., 2021). In addition, the length of the state features represented by the disjunctive graph is usually large and feature redundancy may exist, which need much more computation cost. Moreover, the state features of the disjunctive graph such as node features in these methods are handcrafted. However, the process of acquisition of handcrafted features is a trial-and-error cycle which requires domain knowledge and massive extra experiments on feature selection. Because on the one hand, the experts need to design more and more state features to provide higher semantic information. On the other hand, the number of state features is required to stay short to avoid too much computation time. Therefore, extra feature selection experiments are required to obtain the most effective state features.

For the same aim to tackle the variability in the size of scheduling tasks, the Long Short-Term Memory (LSTM) which belongs to the type of RNN, is used to map the input with arbitrary length into the fixed-length embeddings in Monaci et al. (2021) and they obtain good average results compared with the standard mixed-integer programming solver CPLEX from IBM. However, the encode-decode mechanism which compress all the information into a fixed-length vector makes it hard to cope with very long input (Bahdanau et al., 2014). To alleviate this condition, the attention mechanism which encodes the most important instead of all features into a single fixed-length vector, is utilized in deep neural networks. In Song et al. (2022), Zeng et al. (2022a) and Yang (2022), Graph Attention Network (GAT) is employed to obtain a fixed-length vector from a disjunctive graph and the other parts are similar to that of GNN-based DRL scheduling methods. The Transformer (Vaswani et al., 2017) which is another popular attention mechanism-based model in NLP and CV fields, is also applied to solve the JSSP. A simplified Transformer is shown in Zhao et al. (2022) to extract state features in vector form and A DGERD Transformer is proposed in Chen et al. (2022) where the graph-embedding method is first to extract state features directly from the disjunctive graph and the attention mechanism is then utilized to help agents to learn long-range dependencies better. Event though Zhao et al. (2022) have obtained good results, they only tested the instances with small sizes while Chen et al. (2022) evaluated their method on large-scale benchmarks, but their results are far from the optimal solution and even cannot compete with PDR methods.

Table 1
Comparison of different DRL scheduling methods.

State	Action	Reward	Policy	Optimizer	Reference
DG	Operation	Make-span differences	GE+MLP	PPO	Zhang et al. (2020)
DG	Operation	NO. waiting jobs	GE+MLP	PPO	Park et al. (2021b)
DG	Operation	Make-span	GE+Transformer	SGD	Chen et al. (2022)
DG	Operation	Make-span differences	GAT+MLP	PPO	Song et al. (2022)
DG	Operation	Make-span	GE+MLP	AC	Elsayed et al. (2022)
DG	Operation	Make-span	GNN+softmax	PPO	Lei et al. (2022)
DG	PDR	Machine utilization	Attention+FFN	D3QPN	Zeng et al. (2022b)
Graph	Operation	Make-span	GAT+MLP	PPO	Park et al. (2021a)
Matrix	PDR	Machine utilization	CNN	D3QNPR	Han and Yang (2020)
Matrix	PDR	Make-span	CNN+MLP	DDPG	Liu et al. (2020)
Matrix	Operation	Machine utilization	MLP	PPO	Wang et al. (2021)
Matrix	Operation	Make-span	LSTM	PPO	Monaci et al. (2021)
Variable	Operation	Schedule area	MLP	PPO	Tassel et al. (2021)
Variable	Operation	Delay time of jobs	MLP	DQN	Turgut and Bozdogan (2020)
Variable	Operation	Make-span	MLP	DLDQN	Luo et al. (2021)
Variable	PDR	Tardiness	MLP	DQN	Zhao et al. (2021)
Variable	CDR	Machine utilization	MLP	DQN	Luo (2020)
Variable	Operation	Machine idle time	Transformer	PPO	Zhao et al. (2022)

3. Proposed method

Reinforcement learning problems are usually modeled as Markov decision processes (MDPs), which can be represented as a five tuple $\langle S, A, P, R, \gamma \rangle$. The S represents the set of all observable states; A represents the set of all actions; P is the state transition function, which represents the probability of transitioning from the current state to the next state after executing action a ; R represents immediate reward and γ is a reward discount factor that represents the degree of impact of future rewards on current rewards.

In this section, we describe the overall DRL framework for DJSP. The scheduling process is thought as a single-agent RL problem based on MDP. As is illustrated in Fig. 1, the scheduling policy network is modeled by the MLP network and the Softmax function. At each scheduling step, the scheduling agent chooses a PDR according to the scheduling policy and the job shop environment gives the next state and reward for this selection. The PPO agent collects these data trajectories to learn and update the actor and critic networks until the result is convergent or reach the time limitation.

This framework is based on an actor-critic architecture where the scheduling policy network constructs the actor network and the critic network has the same structure with the actor network. The key parts of the entire framework include (1) a new state representation based on feasible solution matrix which records the scheduling order of job operations on machines; (2) the action space based on the paired PDR; (3) a newly designed reward function derived from the machine idle time; (4) a hybrid priority experience replay method for the PPO.

3.1. State representation based on feasible solution matrix

As one of the most important part of MDP, the state representation defines the form of features of the scheduling environment and determines the size of the state space, which has a great effect on the computation time of state features and scheduling policy. The previous researches on the state representations of JSSP are mainly focus on disjunctive graph (Zhang et al., 2020; Park et al., 2021b; Chen et al., 2022; Song et al., 2022; Elsayed et al., 2022; Lei et al., 2022; Zeng et al., 2022b), feature matrix (Han and Yang, 2020; Liu et al., 2020; Wang et al., 2021; Zhou et al., 2020) and handcrafted feature vectors (Tassel et al., 2021) or state variables (Turgut and Bozdogan, 2020; Luo et al., 2021; Zhao et al., 2021; Luo, 2020; Zhao et al., 2022).

The disjunctive graph integrates the operation dependencies and machine status of JSSP, providing critical structural information for scheduling decisions. However, the node features are manually designed and more extra information such as the assignment indicator (Zhang et al., 2020; Park et al., 2021b; Chen et al., 2022; Song et al., 2022; Elsayed et al., 2022; Lei et al., 2022; Zeng et al., 2022b), waiting

time (Park et al., 2021b; Zeng et al., 2022b), the lower bound time of completion (Zhang et al., 2020; Elsayed et al., 2022; Lei et al., 2022) and processing time (Park et al., 2021b; Song et al., 2022; Lei et al., 2022; Zeng et al., 2022b), is required in the disjunctive graph.

For the matrix representation of state features, each matrix represents one meaningful feature of the scheduling environment such as machine utilization matrix in Han and Yang (2020), the assignment matrix in Liu et al. (2020) and the processing time matrix (Han and Yang, 2020; Liu et al., 2020; Wang et al., 2021). The feature matrix in these methods is usually thought as one of the channels of a color image so that a lot of feature extraction methods in computer vision fields such as CNN can be employed for subsequent work. However, the feature matrices in these approaches are still designed manually and the channels are limited in a color image.

When represented by handcrafted feature vectors/variables, more and more features are designed no matter whether they have understandable meanings or not. For example, the processing time (Tassel et al., 2021; Zhao et al., 2022), the ratio of each job's processed operation time to each job's total operations time (Luo et al., 2021) and the average utilization of all machines (Zhao et al., 2021; Luo, 2020) are all manually designed. However, how many state features are really needed and what kind of state feature is effective in the state representation remain unsolved, which needs massive time to perform extra experiments on feature selection.

From the aforementioned representations, the state features of scheduling environment are mainly handcrafted which needs a myriad of specialized domain knowledge. On the other hand the state features require extra static job and machine information such as the operation processing time. In this paper, a novel simple state representation is proposed to avoid handcrafted feature designation and feature selection, which requires less static job and machine information. As illustrated in Fig. 1 (which take a feasible solution of a 6x6 instance as an example), the state feature is represented by a feasible solution matrix. The feasible solution matrix is a 2-dimensional vector with a height of job number and a width of operation number of each job (which also equals the number of machines). The scheduling order number is stored in the feasible solution matrix and the values are scaled by the number of job operations to be in the range [0,1]. The state representation is expressed in Eq. (1).

$$\text{Solution}[job_i][machine_j] = n / N_{ops} \quad (1)$$

where the job_i is the job selected by the scheduling policy; the $machine_j$ is the machine needed by job_i ; n is the n th selection of the scheduling policy; the N_{ops} is the total number of job operations.

Compared with the state representation in previous researches, our state representation has three advantages: (1) the calculation of the feasible matrix is simple and the length of this state representation is

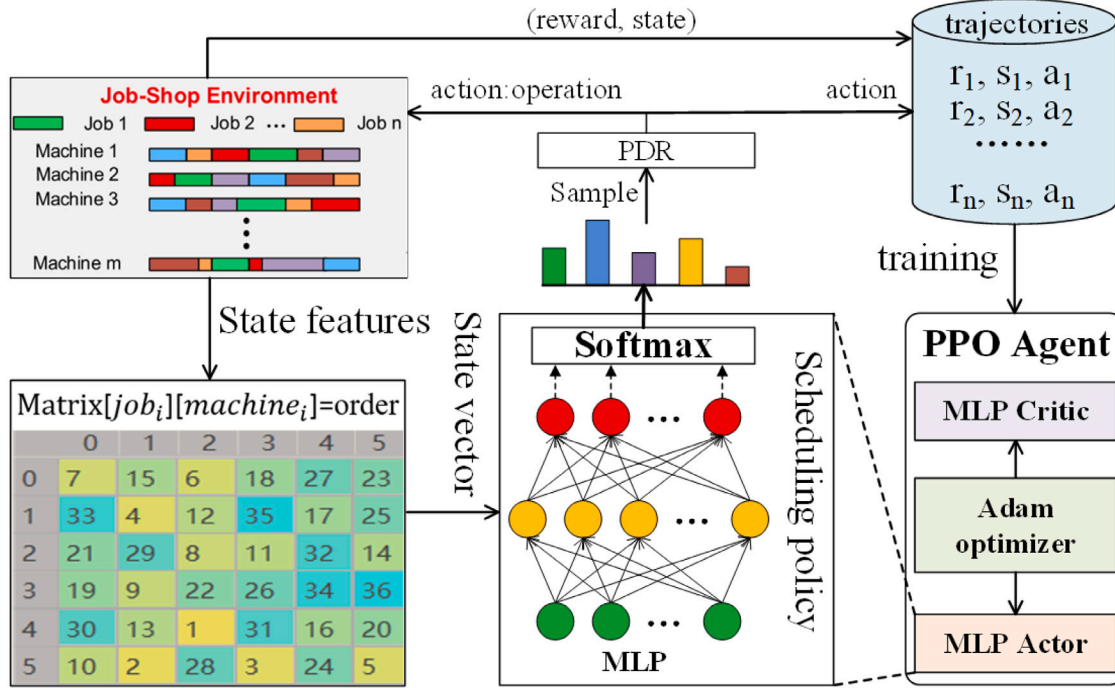


Fig. 1. The entire DRL framework for dynamic job-shop scheduling problems.

less than the other matrix representation, which reduce the computation time of environment and deep neural networks of DRL agent, respectively; (2) our state representation directly employs the form of the feasible matrix, which need no extra specialized knowledge to design new state features; (3) our state representation contains dynamic information of scheduling jobs and machines and the history scheduling information, which is more independent with the static job shop information such as operation processing time, leading to better generality of representation for instances with different operation processing time.

3.2. The action space based on paired PDR

The definition of action space in previous researches is mainly focused on the job operation (Zhang et al., 2020; Elsayed et al., 2022; Park et al., 2021b; Chen et al., 2022; Lei et al., 2022; Song et al., 2022; Wang et al., 2021; Luo et al., 2021; Tassel et al., 2021; Zhao et al., 2022), the PDR (Zeng et al., 2022b; Han and Yang, 2020; Liu et al., 2020; Zhao et al., 2021; Luo, 2020) and waiting action (Park et al., 2021b) or no-op (Tassel et al., 2021).

When the job operation set is employed as the action space, the size of operation action space is the amount of all jobs because at each decision step only one operation can be performed among all the operations in one job. However, the scheduling policy may encounter illegal action problem where the selected job operation cannot be performed because the machine needed by the selected job operation is occupied by other job operation. To address this problem, operation mask (Huang and Ontanon, 2022) is applied to the output of a neural network to transform values of illegal action into a very big negative number (e.g. $-1e8$). As a result, the resample probability of the illegal action is very close to 0 after outputting from the Softmax function. However, this mask information is related to the size of jobs which make it hard to generalize to other instances with different sizes.

For the PDR action space, a PDR is selected at each time step and a job operation is subsequently derived from this PDR. If there are assignable jobs, the derived operations can be performed on the needed machines, which means that there is no illegal action problem. However, the problems of how many PDR should be selected to construct the state space and what kind of PDR need to be selected to ensure

the diversity of the derived job operations from PDR have not been resolved.

We have to notice that, active no-op which schedules no operation and directly advances to the next time step even though there are assignable jobs, sometimes outperforms any assignable operations. As is illustrated in Fig. 2, the scheduling make-span of the left solution is better than the right one because of the no-op instead of J1 at time 5. As a result, no-op sometimes is considered when constructing action space (Tassel et al., 2021). However, the no-op is not considered in our method because although active no-op sometimes can achieve better performance, it does not perform well most of the time, which make it complex to learn. On the other hand, since the reward of no-op in our method is always negative, the scheduling agent will stop using no-op as the number of iterations increases.

In our proposed method, twelve PDR are selected to build the action space for the sake of simplicity and ease of generalization. Four out of twelve PDR are directly selected from Zhang et al. (2020) which contains Shortest Processing Time (SPT), Most Work Remaining (MWKR), Minimum ratio of Flow Due Date to Most Work Remaining (FDD/MWKR) and Most Operations Remaining (MOPNR). The Longest Remaining Machine time not including current operation processing time (LRM) is selected from Han and Yang (2020) due to its better performance. The First In First Out (FIFO) is selected because it is widely used in various scheduling problems. The PDR in previous researches usually appear in pairs and in our proposed method, the PDR with opposite priority are also parts of the action space to increase the diversity of the derived job operations. The definitions of these priority dispatching rules are summarized as follows:

- SPT (Select the job with the shortest processing time): $\min Z_{i,j} = p_{i,j}$
- LPT (Select the job with the longest processing time): $\max Z_{i,j} = p_{i,j}$
- MWKR (Select the job with the most work remaining): $\max Z_{i,j} = \sum_j^{n_i} p_{i,j}$
- LWKR (Select the job with the least work remaining): $\min Z_{i,j} = \sum_j^{n_i} p_{i,j}$

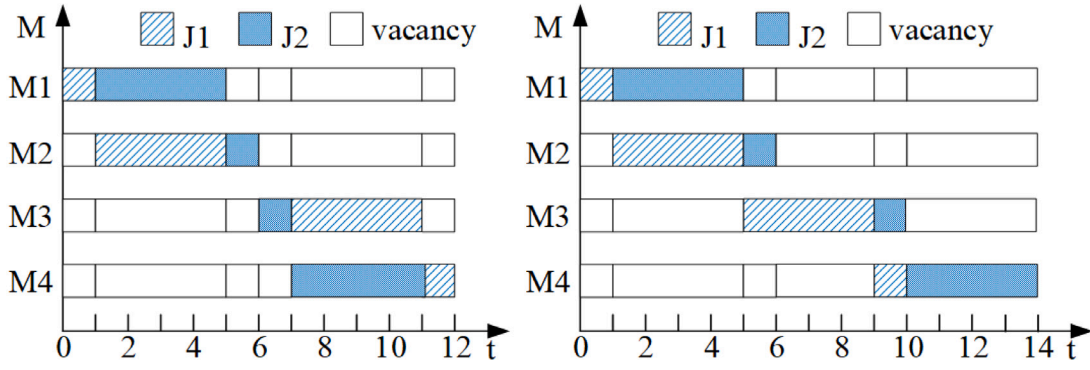


Fig. 2. Examples of no-op in the scheduling environment.

- FDD/MWKR (Select the job with the minimum ratio of Flow Due Date to Most Work Remaining): $\min Z_{i,j} = \sum_1^j p_{i,j} / \sum_j^{n_i} p_{i,j}$
- FDD/LWKR (Select the job with the maximum ratio of Flow Due Date to Least Work Remaining): $\max Z_{i,j} = \sum_1^j p_{i,j} / \sum_j^{n_i} p_{i,j}$
- MOPNR (Select the job with the most remaining operations): $\max Z_{i,j} = n_i - j + 1$
- LOPNR (Select the job with the least remaining operations): $\min Z_{i,j} = n_i - j + 1$
- LRM (Select the job with the Longest Remaining Machine time not including current operation processing time): $\max Z_{i,j} = \sum_{j-1}^{n_i} p_{i,j}$
- SRM (Select the job with the Shortest Remaining Machine time not including current operation processing time): $\min Z_{i,j} = \sum_{j-1}^{n_i} p_{i,j}$
- FIFO (Select the job with the first release time): $\max Z_{i,j} = t - Rt_i$
- LIFO (Select the job with the last release time): $\min Z_{i,j} = t - Rt_i$

Where $Z_{i,j}$ is the priority index of operation $O_{i,j}$. $p_{i,j}$ is the processing time of operation $O_{i,j}$. n_i is the number of operations for job J_i . j is the operation order of $O_{i,j}$. t is the current time and Rt_i is the release time of J_i .

3.3. Reward function based on the area of machine idle time

The objective of a RL agent is to maximize the accumulated reward from the environment while the objective of JSSP is usually to minimize the maximum make-span across all the jobs. The definition of the reward function in a DRL scheduling method should consider both two aspects. However, minimize the maximum make-span might lead to a sparse reward issue because it can only be obtained at the end of the entire scheduling process. To solve this problem, other scheduling factors related to make-span are considered in reward function such as machine utilization (Zeng et al., 2022b; Han and Yang, 2020; Wang et al., 2021), tardiness (Luo, 2020; Zhao et al., 2021) or delay time of the jobs (Turgut and Bozdog, 2020) and scheduled area (Tassel et al., 2021).

In this paper, we propose a clear and intuitive reward function (Eq. (2)) based on the idle time of all scheduling machines. This reward function is similar to that in Tassel et al. (2021) which calculates the difference between the duration of the allocated operations and the introduced holes after each action. However, there is a lack of clear mathematical relationship between the reward function and the maximum make-span in their DRL scheduling method. Our idea is motivated by the fact that the total time scheduling area is equal to the sum of total processing time of all jobs and the idle time of all scheduling machines, which also equals the maximum make-span multiplied by the number of machines. As is demonstrated in Fig. 2, the scheduling time area is composed of the total processing time of all jobs (shaded area) and the time vacancy on all machines (white color area).

This relationship between the accumulated reward and the maximum make span is expressed in Eq. (3).

$$reward(s_t, a_t) = - \sum_M vacancy_m(s_t, s_{t+1}) \quad (2)$$

where s_t is the current state and s_{t+1} is the next state after applying action a_t ; $vacancy_m(s_t, s_{t+1})$ is a function returning the idle time of all machines in M while transitioning from state s_t to s_{t+1} .

$$R = -b = a - (a + b) = a - S = a - M * makespan \quad (3)$$

where R is the total reward, a is the total processing time of all jobs which is constant, b is the total vacancy time on all machines, S is the scheduling time area and M is the total number of machines.

Since the total processing time of all jobs is constant, we can conclude from Eq. (3) that in our reward function, the total reward and the maximum make-span are negatively linearly dependent and the coefficient is the number of machines. That is, minimizing the maximum scheduling make-span is equivalent to maximizing the total reward, resulting in the consistency between scheduling and learning objective.

3.4. A hybrid prioritized experience replay method for PPO

The scheduling policy in early RL scheduling methods is usually expressed as arrays or tables. Due to the lack of representation ability, traditional RL scheduling methods can only solve small-scale scheduling tasks with low-dimensional state and action space. For large scale scheduling tasks which usually have a higher dimensional state space, the RL will face the dimension disaster problem. The achievements in image classification, NLP and other fields have shown that deep learning has powerful representation capabilities. Moreover, deep neural networks such as CNN, RNN and MLP, have been proved to be general function approximators, which can be used to approximate value function and policies in complex tasks with high-dimensional input.

In this paper, the scheduling policy is represented by the MLP network with two hidden layers and a Softmax function (see Eq. (4)). The MLP network is used to extract the state features and obtain a scalar score for each action. The Softmax function is applied to output a distribution over the computed score. The parameters of our scheduling policy are learned by an actor-critic clipped PPO whose loss function is expressed in Eq. (5). The actor network is constructed by the scheduling policy and the critic network is represented by the MLP network which has the same hidden layers with the actor network.

$$p(a_t|s_t) = Softmax(MLP_{\pi_{\theta}}(s_t)) \quad (4)$$

$$L(\theta) = E_t[\min(r_t(\theta)A_t, clip(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)] \quad (5)$$

Where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, A_t is an estimator of the advantage function at time step t and ϵ is a hyper parameter.

In stochastic gradient-based DRL method, experience replay is employed to satisfy the i.i.d. assumption and use the rare useful experience for more than just a single update. Traditional experience replay method sample the experience evenly during the training, regardless of their significance. Prioritized experience replay (PER) replays the experience samples according to their importance and frequently replay those experience samples with high expectations that use temporal-difference error (TD-error) to measure the level of priority. A proportional prioritization stochastic sampling method (the probability of sampling is defined in Eq. (6)) is introduced in Schaul et al. (2016), which interpolates between pure greedy prioritization and uniform random sampling. The importance-sample weights (IS weights) expressed in Eq. (7) is used in this method to correct the sample distribution because the estimated distribution of values will have bias with respect to the true distribution as the training updates increase. Besides, greedy prioritization focus on a small subset of the experience where errors shrink slowly, especially when using function approximation. This lack of diversity that makes the policy networks prone to over-fitting.

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (6)$$

where $P(i)$ is the priority of transition i which is greater than 0; the exponent α determines the priority level, with $\alpha = 0$ corresponding to the uniform case; $p_i = \text{abs}(TD) + \delta$ is proportional prioritization, where δ is a small positive number to enable the experience samples with 0 TD-error to be sampled.

$$w_i = \left(\frac{1}{N} \times \frac{1}{P_i}\right)^\beta \quad (7)$$

where N is the length of buffer and β is a parameter which is changing from its initial value to 1 and $\beta = 1$ means fully compensates for the non-uniform probabilities P_i .

In this paper, we propose a hybrid experience replay method which integrates the uniform and prioritization sampling. At each update, all the experience samples are firstly replayed evenly and the priority of experience samples are calculated according to Eq. (6). Then, the PER is performed. To demonstrate the potential effectiveness of prioritizing replay by TD-error for JSSP, we compare the performance of different PER variants on the benchmark instance LA21. Specifically, we divide the TD-error samples into three categories: positive, negative and absolute TD-error samples to explore their respective effects on the scheduling performance. Besides, we consider the distribution correction using IS weights for actor network, critic network and both networks of PPO, respectively. Moreover, for positive TD-error samples, we separately anneal the batch size from 1 to two times of instance scale using linear, power and step function. For each PER variants, we perform the experiments on the same instance for five times and we set the time limitation in half an hour.

As is shown in Fig. 3, the original PPO without PER cannot converge within the time limitation but shows good exploration ability to get smaller make-span. As a result, the most urgent need for us is to obtain a convergent scheduling policy. We can see from the performance of these PER variants that positive TD-error samples can help to improve the convergence efficiency but to some extent it will be unfavorable for the exploration, samples with absolute TD-error are conducive to improving the exploration ability and negative TD-error samples may impair the convergence efficiency of our scheduling policy. The distribution correction using IS weight for the actor network is not needed, since the loss function of actor network has been clipped and all the experience samples are replayed at each update. Besides, there are small differences between the annealing batch size methods. Since the parameter of IS weight is adjustable, whether using IS weight to correct the distribution bias for critic network or not, has no obvious effect on the scheduling performance. As a result, we choose the positive TD-error prioritization method with power annealing batch size to replay the experience samples due to the simpler implementation.

The detailed training process is described in Algorithm 1, which includes two aspects: data collection (Lines 5–16) and policy learning (Lines 17–26). When collecting training data, we generate T independent complete trajectories of scheduling and store them in the memory buffer M . On the policy learning stage, we divide these data randomly into batches whose length is related to the scale of instances, and update the actor and critic networks. Then, the priority of experience samples are calculated based on the TD error and PER is performed to update the actor and critic networks again. In each learning time, the scheduling agent learns from experience samples for K times. The training process will stop until the episode reaches the maximum iterations or the result is convergent or reaches the time limitation. We define that the result converges if 30 consecutive scheduling results are the same in make-span and the training time is limited in one hour.

Algorithm 1: The training process of scheduling agent for DJSP based on PPO with PER

```

Initialize maximum episode  $N$ , discount factor  $\gamma$ , the number
of trajectories  $T$ ;
Initialize buffer  $B$ , buffer  $M$ , batch size  $b$ , update steps  $K$  and
resample batch size  $Rb$ ;
Initialize actor, critic networks and corresponding optimizers;
while  $e \leq N$  do
    while  $t \leq T$  do
        state = env.reset();
        while true do
            action, actionProb = selectAction(state);
            nextSstate, reward, done = step(action);
            Store transition(state, action, reward, actionProb) in
                 $B$ ;
            state = nextState;
            if done then
                Calculate discounted reward using  $\gamma$  and replace
                    reward with it;
                Add  $B$  to  $M$  and clear  $B$ ;
                break;
            end
        end
    end
    while  $k \leq K$  do
        Divide  $M$  randomly into  $S$  subsets with  $b$  transitions;
        while  $s \leq S$  do
            Optimize and update the actor and critic networks
                using data from each subset;
            Calculate the priority of experience samples
                according to TD error;
        end
        Calculate the resample batch size  $Rb$  using power
            annealing method;
        Resample  $Rb$  transitions based on the priority of
            experience samples;
        Optimize and update actor and critic networks,
            respectively;
    end
    if the result is convergent or time out then
        break;
    end
end

```

4. Experiments

In this section, in order to evaluate the performance of our DRL scheduling methods, a group of static experiments are performed on public JSSP benchmarks with various sizes and the results are compared with three different DRL scheduling methods from the recent literature. Based on three rescheduling mode, dynamic experiments are

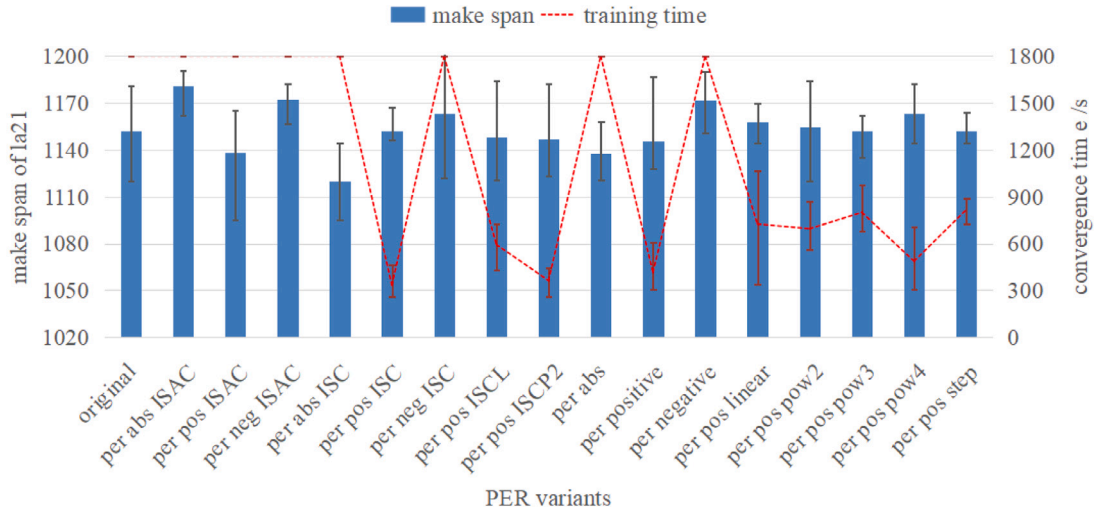


Fig. 3. The make-span and the convergence time of instance la21 for PPO with PER variants.

Table 2

Benchmarks.

Benchmarks	Source
ft10 (10 × 10)	Fisher (1963)
la01 (10 × 5), la06 (15 × 5), la11 (20 × 5), la16 (10 × 10), la21-la25 (15 × 10), la26-la30 (20 × 10), la31-la35 (30 × 10), la36-la40 (15 × 15)	Lawrence (1984)
ta21-ta22 (20 × 20), ta31-ta32 (30 × 15), ta41-ta42 (30 × 20), ta51-ta52 (50 × 15)	Taillard (1993)

designed to evaluate the generality of our DRL scheduler for uncertainties such as uncertain operation processing time and the training details such as the maximum make-span, convergence episodes and convergence time of our DRL scheduler are compared and demonstrated.

4.1. Benchmark instances

In this paper, we evaluate our DRL scheduling method on standard benchmark instances of various sizes which are listed in Table 2. On the other hand, in order to validate the generality of our method, instances with uncertainties for rescheduling are generated based on the standard benchmarks with different sizes.

The uncertainty ratio and the degree of uncertainty of operation processing time are considered for the generation of new instances. Specifically, the ratio of uncertain operations is separately set to 25%, 50%, 75% and 100% and the degree of uncertainty is measured by Normal and Uniform distribution with three level deviations, respectively. The parameters of generating the new instances are listed in Table 3. We divide the generated instances into two categories according to the distribution to explore the effect of different distribution on the performance of DRL method. In each datasets, five standard instances are used to generate new instances where the machine information of standard instances stays unchanged and the processing time of job operations follows Normal or Uniform distribution. The mean values of the generated processing time of job operations are the same as the operation processing time of standard benchmark instances. As a result, for each distribution datasets, the total number of newly generated instances is:

4 standard instances × 4 kinds of uncertainty ratios × 3 kinds of deviations = 48 instances

4.2. Models and configurations

In our DRL model, the actor and critic networks of PPO are both implemented by MLP networks with 2 hidden layers. The dimension of each hidden layer network equals the scale of a scheduling instance. In the training process, for each scheduling instance, we train the

policy network for at most 4000 iterations, each of which contains 9 independent trajectories (i.e., complete scheduling process of instances) and we use a dynamic batch size which equals 2 times of the scale (the total number of operations of all jobs) of an instance. For PPO, we set the epochs of updating network to 10 and the clipping parameter epsilon to 0.2. We set the discount factor γ to 0.999 and use the Adam optimizer with constant learning rate $1e-3$ and $3e-3$ for actor and critic network, respectively. When using PER, the parameter for the prioritization sampling probability is set to 0.6 and the resampling the batch size anneals from 1 to two times of instance scale using a power function with parameter 2.

The hyper parameters of the number of trajectories, batch size and the dimension of hidden layers of actor network are all carefully selected and the other parameters are widely used in the literature. The experiments run on a laptop equipped with Windows 10 64 operating system 8G RAM, Intel Core i7-9750H 2.60 GHz CPU. Other parameters follow the default settings in PyTorch and our code is available at [github](#).

4.3. Static experiments and results analysis

To evaluate the performance and the generality of our DRL scheduling method, static experiments are performed on standard benchmark instances with various sizes from 10x5 to 50x15 which are shown in Table 2. We train these instances independently for 5 times and the average results are recorded. As is shown in Table 4, we compare our results with three recent different DRL scheduling methods and PDR scheduling methods on JSSP. The scheduling results of the compared DRL methods are directly get from the literature and the “-” in Chen et al. (2022) column means that they did not run their method on these instances. The scheduling results of PDR method are obtained in our environment and only the best results of all twelve PDR scheduling methods are printed in Min PDR column. The values in the OPT column are the known optimal scheduling results. The size of the instances is given by the number of jobs × the number of machines and the best make-span results are printed in boldface. Besides, in order to show the

Table 3
The parameters of generated instances.

Base instances	Distribution	Deviation	Uncertainty ratio
ft10 (10 × 10)	Normal	Sigma = 1, 2, 3	25%, 50%, 75%, 100%
	Uniform	Bound = ± 10%, 20%, 30%	25%, 50%, 75%, 100%
la26 (20 × 10)	Normal	Sigma = 1, 2, 3	25%, 50%, 75%, 100%
	Uniform	Bound = ± 10%, 20%, 30%	25%, 50%, 75%, 100%
la31 (30 × 10)	Normal	Sigma = 1, 2, 3	25%, 50%, 75%, 100%
	Uniform	Bound = ± 10%, 20%, 30%	25%, 50%, 75%, 100%
ta41 (30 × 20)	Normal	Sigma = 1, 2, 3	25%, 50%, 75%, 100%
	Uniform	Bound = ± 10%, 20%, 30%	25%, 50%, 75%, 100%

Table 4
Make-span comparison on standard benchmark instances with different sizes.

Size	Instance	OPT	Chen et al. (2022)	Han and Yang (2020)	Park et al. (2021a)	Min PDR	Ours	Score
10 × 5	la01	666	–	666	692	680	666	0.00%
15 × 5	la06	926	–	926	971	926	926	0.00%
20 × 5	la11	1222	–	1222	1319	1254	1222	0.00%
10 × 10	la16	945	–	980	1054	1047	1027	−4.80%
15 × 10	la21	1046	–	1162	1261	1230	1147	1.29%
15 × 10	la22	927	–	1021	1207	1060	997	2.35%
15 × 10	la23	1032	–	1053	1145	1152	1039	1.33%
15 × 10	la24	935	–	1029	1088	1085	1015	1.36%
15 × 10	la25	977	–	1067	1117	1112	1050	1.59%
20 × 10	la26	1218	–	1327	1458	1386	1283	3.32%
20 × 10	la27	1235	–	1397	1516	1415	1353	3.15%
20 × 10	la28	1216	–	1386	1357	1472	1305	3.83%
20 × 10	la29	1152	–	1323	1320	1343	1288	2.42%
20 × 10	la30	1355	–	1417	1490	1534	1395	1.55%
30 × 10	la31	1784	–	1854	1906	1810	1786	1.65%
30 × 10	la32	1850	–	1900	1850	1884	1850	0.00%
30 × 10	la33	1719	–	1782	1731	1794	1720	0.64%
30 × 10	la34	1721	–	1880	1784	1856	1746	2.13%
30 × 10	la35	1888	–	1941	1969	2039	1896	2.32%
15 × 15	la36	1268	–	1355	1449	1396	1357	−0.15%
15 × 15	la37	1397	–	1540	1653	1584	1511	1.88%
15 × 15	la38	1196	–	1348	1444	1358	1327	1.56%
15 × 15	la39	1233	–	1357	1430	1405	1343	1.03%
15 × 15	la40	1222	–	1336	1350	1358	1314	1.65%
20 × 20	ta21	1642	2145.63	1952	1921	1964	1876	2.34%
20 × 20	ta22	1600	2015.89	1870	1844	1868	1794	2.71%
30 × 15	ta31	1764	2382.63	1986	2055	2134	1965	1.06%
30 × 15	ta32	1784	2458.52	2135	2268	2163	2096	1.83%
30 × 20	ta41	2005	2541.22	2450	2572	2499	2398	2.12%
30 × 20	ta42	1937	2762.26	2351	2397	2401	2305	1.96%
50 × 15	ta51	2760	3762.6	3263	3382	3442	3155	3.31%
50 × 15	ta52	2756	3511.2	3229	3231	3263	3056	5.36%

improvement of our DRL method, the improvement score is calculated according to Eq. (8).

$$score = \frac{makeSpan_{bestKnown} - makeSpan_{ours}}{makeSpan_{bestKnown}} \times 100\% \quad (8)$$

Where the $makeSpan_{bestKnown}$ is the best result obtained by the compared DRL methods and the best results of all 12 PDR performed in our environment. The $makeSpan_{ours}$ is the average scheduling results of our DRL models.

It is usually difficult to compare different DRL scheduling methods because of their different goals/settings/algorithms. There are two main training modes to obtain a scheduling policy due to different goals. One training mode is that the scheduling agent is trained on randomly generated instances instead of benchmark instances and is tested on unseen instances because the goal is to obtain a general scheduling policy beyond present instances. The other training mode is similar to the traditional instance-specific optimization method which does not reuse the knowledge gained from other instances. Chen et al. (2022) and Park et al. (2021a) follow the former training mode while Han and Yang (2020) use the later training mode. In this paper, we use the same training mode with Han and Yang (2020). Nevertheless, they used a different DRL method called Dueling Double DQN with Priority Experience Replay (D3QNPR) which is an off-policy, value-based algorithm. Besides, their training is limited to 8000 episodes instead of a time limit.

We compared our scheduling agent with three different DRL baseline model from Chen et al. (2022), Park et al. (2021a) and Han and Yang (2020). The scheduling results in Park et al. (2021a) and Han and Yang (2020) are SOTA in DRL scheduling methods and the model in Chen et al. (2022) is based on the popular Transformer which is recent and competing. As is illustrated in Table 4, our DRL scheduling method obtains smaller make-span in 30 out of 32 instances than that in Han and Yang (2020) and outperforms the other two DRL methods and all twelve PDR methods on all the selected instances. The improvement score shows no obvious difference as the size of scheduling tasks changes, which indicates that the stability of our method across scheduling tasks with different sizes. It is noted that the DRL scheduling method in Chen et al. (2022) shows significant worse performance than the PDR methods which are simpler and faster. This indicates that the DRL scheduling methods have not always been better than PDR methods.

We then show the convergence performance of our DRL scheduling method. In Section 3.4, we defined the convergence condition and other training stop condition that the training step reaches 4000 episodes or one hour is used up. In each episode, nine trajectories are collected to train our DRL model so that the maximum number trajectories is 36 000. As is demonstrated in Fig. 4, our DRL model on the instances from la01 to la40 are convergent in an hour and the other are not convergent due to the time limitation. The average convergence time



Fig. 4. The training trajectories and training time of our DRL method.

for small-scale instances (10×5 , 15×5 , 20×5 , 10×10 and 15×10) is less than 500 s, for middle-scale instances (20×10 and 15×15) is less than 1800 s and for large-scale instances (30×10) is less than 2800 s. As the scale of instances increases, the time for collecting a trajectory and updating the policy grows obviously.

4.4. Dynamic experiments and results analysis

Various uncertainties such as insertion of new jobs, change of operation processing time, machine breakdown and rush orders are ubiquitous in the practice, which have been considered in real-world scheduling. These uncertainties may cause deviations from the predictive schedule over the process of execution and rescheduling is needed to adapt to these uncertainties. There are three rescheduling mode for DRL scheduling methods. The first one is to retrain a complete new scheduling policy on the new instances, which is similar to traditional rescheduling method and does not reuse the knowledge gained before. The second one is to reschedule with a trained scheduling policy which employs the knowledge gained from other instances. The trained scheduling policy stays unchanged during the rescheduling. The final one is an adaptive rescheduling mode. Before rescheduling, a trained scheduling policy is reused and retrained on new instances, which integrates the knowledge gained from old instances and that from new instances. The reused scheduling policy continuously improves for new changes in DJSP.

In this paper, the scheduling environment condition with uncertain operation processing time is considered and the rescheduling process is illustrated in Fig. 5. The complete scheduling process is described as follows. The scheduling policy is first trained using our DRL agent in a virtual environment to get a trained scheduling policy. The real scheduling process is then performed according to the trained scheduling policy. When the operation processing time is changed, the trained policy is reused and retrained on instances with new changes to obtain a new trained policy. The real rescheduling process is finally performed according to the newly trained scheduling policy. In our dynamic experiments for rescheduling, static experiments were first executed on the standard instances in Table 3 to obtain the trained scheduling policies.

We first perform the dynamic experiments on the generated instances following the Normal distribution and we validate our DRL method using all the three rescheduling mode mentioned above. As is shown in Fig. 6, the scheduling results of PDR and the trained scheduling policies are compared, which indicate that the overall performance of the trained policy (red line) is better than all the PDR methods. There

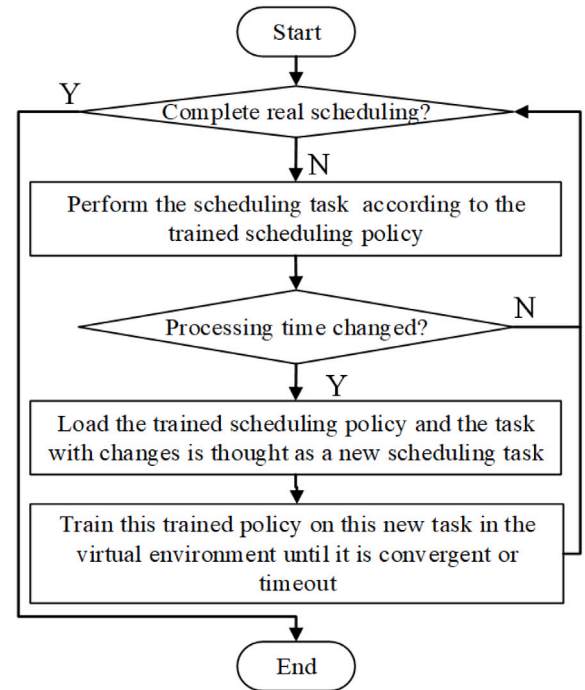


Fig. 5. The rescheduling process flow chart when the operation processing time is changed.

is no obvious difference on the size of the instances, the uncertainty ratio and the deviation degree of operation processing time.

The reused and retrained new policies are evaluated in rescheduling and their training time on new instances is compared. As is demonstrated in Fig. 7, the reused rescheduling policy can obviously reduce the training time on small and medium-scale instances. Besides, the reused scheduling policy tends to be easier to converge than the retrained new policy. For large-scale instances, the reused scheduling policy has no better convergence than the retrained new policy in limited time.

The make-span performance of the PDR, the trained, the reused and the retrained scheduling policies is compared together in Fig. 8. Among the four kinds of rescheduling policies, the PDR methods achieve worse performance on the make-span. The reused policy (the red solid line) is generally better than the trained policy (the black dotted line).

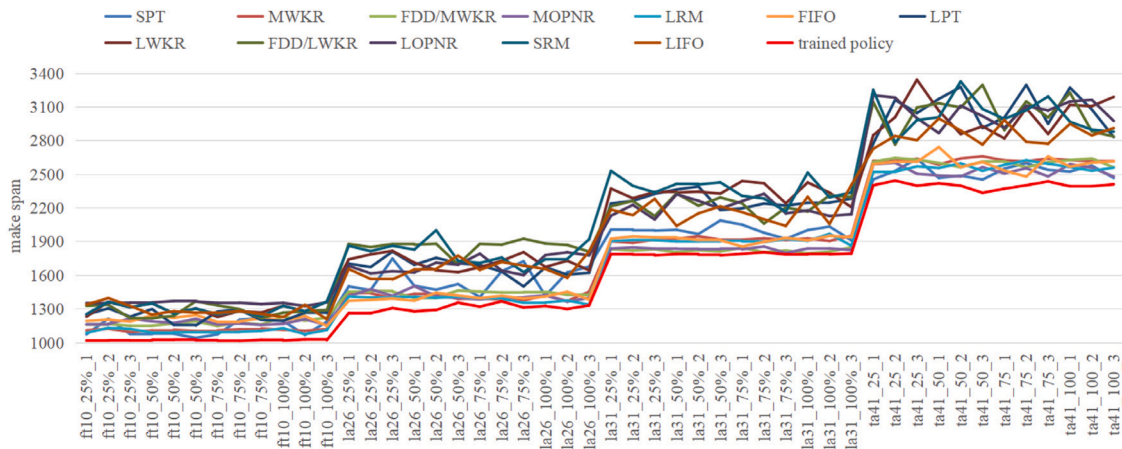


Fig. 6. The make-span results of different PDR and the trained scheduling policies on instances with Normal distribution.

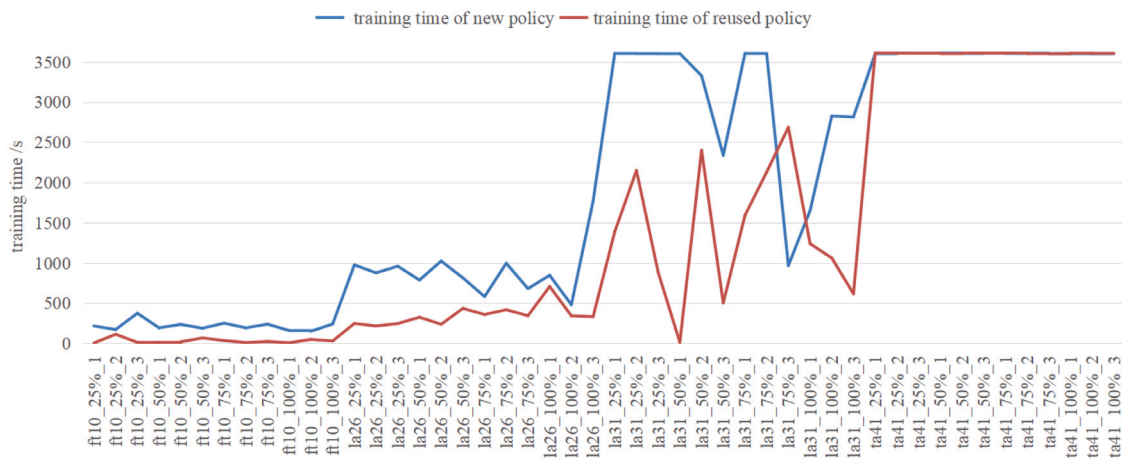


Fig. 7. The convergence time of reused and retrained new scheduling policies on instances with Normal distribution.

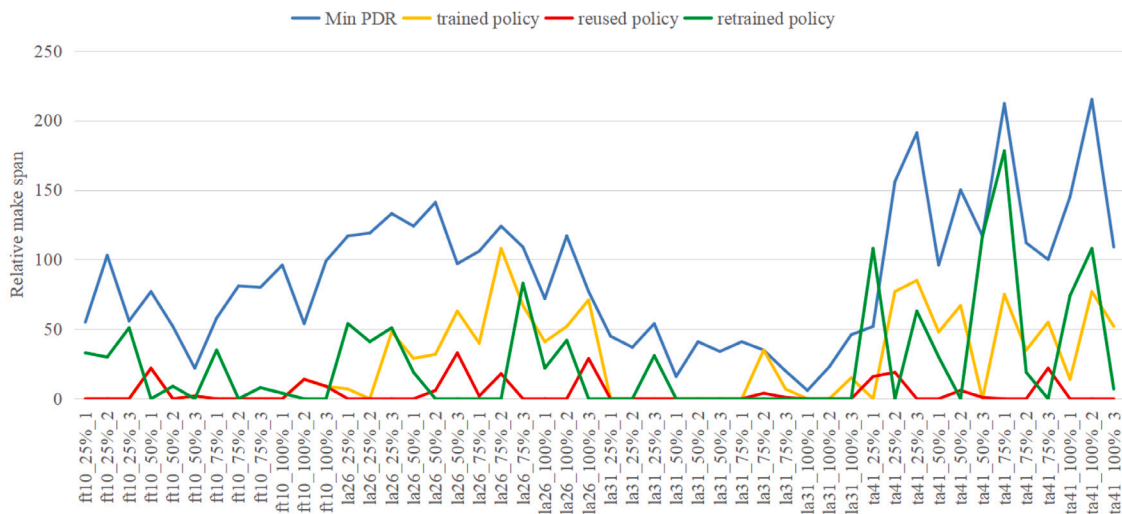


Fig. 8. The make span of four kinds of rescheduling policies on instances with Normal distribution.

Although the retrained new scheduling policy is comparable with the trained and the reused policy, it is not stable on all the instances with different sizes. Surprisingly, it seems that the uncertainty ratio has no effect on the performance of all the four kinds of rescheduling policies.

Since the running time of PDR and the trained scheduling policies for one instance is shorter than one second, the three rescheduling

modes have their own advantages in different scenarios. In order to obtain faster reaction to uncertainties, PDR and the trained scheduling policies are better selections while the reused and the retrained new scheduling policies are suitable for finding more accurate scheduling solution under uncertainty. The reused scheduling policy seems to be the compromised selection for both faster and more accurate

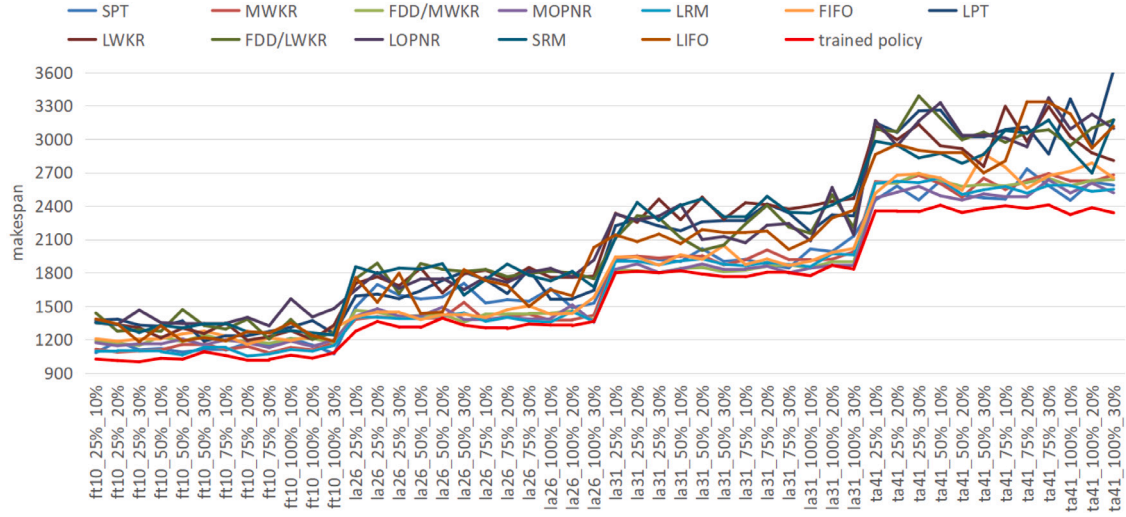


Fig. 9. The make span of PDR of rescheduling policies on instances with Uniform distribution.

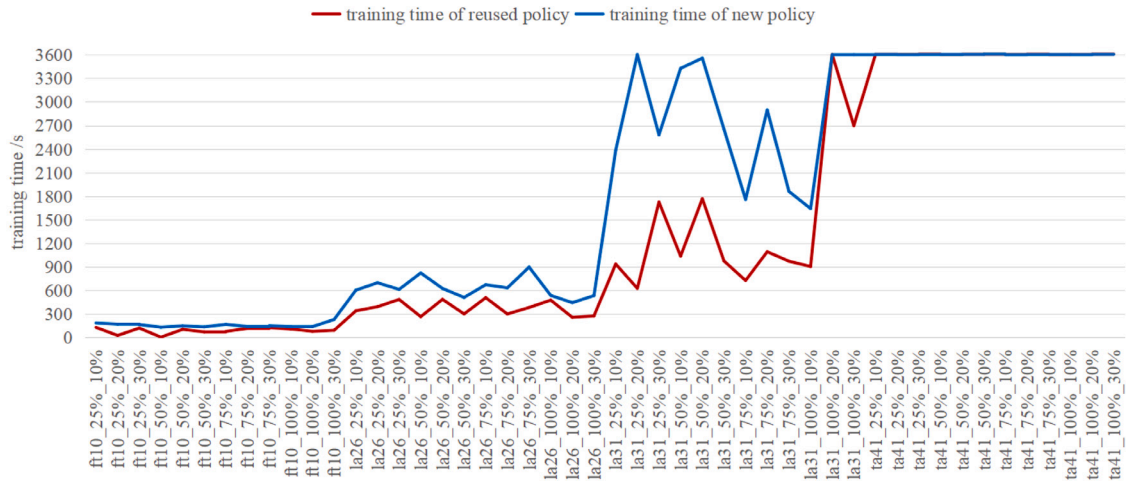


Fig. 10. The convergence time of reused and retrained new scheduling policies for instances with Uniform distribution.

scheduling solution and can continuously evolve when confronting new uncertainties.

The same dynamic experiments are performed on the generated instances whose operation processing time follow the Uniform distributions. As is shown in Figs. 9, 10 and 11, the scheduling results on instances with Uniform distributions are similar to that on instances with Normal distributions. This indicates that when uncertainties occur on the operation processing time, the distribution difference has no effect on the performance of our scheduling policy. Since the output of our scheduling policy is a PDR instead of a direct job operation, the derived job operation may be adjustable as the operation processing time changes. If the output of the scheduling policy is a direct job operation, extra experiments need to be conducted to validate this conclusion.

5. Conclusion and future work

In this paper, we propose a DRL scheduling method for DJSP. Specifically, a novel state representation is proposed based on feasible solution matrix, avoiding the specialized knowledge on feature design and extra experiments on feature selection. Besides, a new reward function is designed based on machine idle time area and the accumulated reward is proved to be negatively linearly dependent with the

maximum make-span of a scheduling task. The improvements on the two aspects provide high-quality learning samples for the scheduling agent. It is convenient to replace the PPO with other kinds of RL methods such as DQN to construct new DRL scheduling methods. To improve the utilization of experience samples and obtain a convergent scheduling policy, a hybrid prioritized experience replay method for PPO is proposed where the positive TD-error samples are selected to replay using the power annealing replay batch size. Static experiments show that our DRL scheduling method significantly outperforms the SOTA DRL methods on classic benchmark instances with different sizes. In the dynamic environment, our scheduling model can significantly reduce the rescheduling time when the operation processing time is uncertain. Moreover, the distribution of operation processing time has no obvious effect on the performance of our scheduling policy.

Although the impact of most uncertainties in scheduling systems are considered to be related to processing time, such as machine failures, differences in worker experience, and changes in logistics, there are some uncertainties related to the scale of scheduling problems, such as randomly arrived jobs, emergency orders, and so on. At present, our method is not suitable for scheduling problems with variant sizes. In the future, we plan to construct a deep neural networks to generate fixed length state features to support DJSP with scale uncertainties.

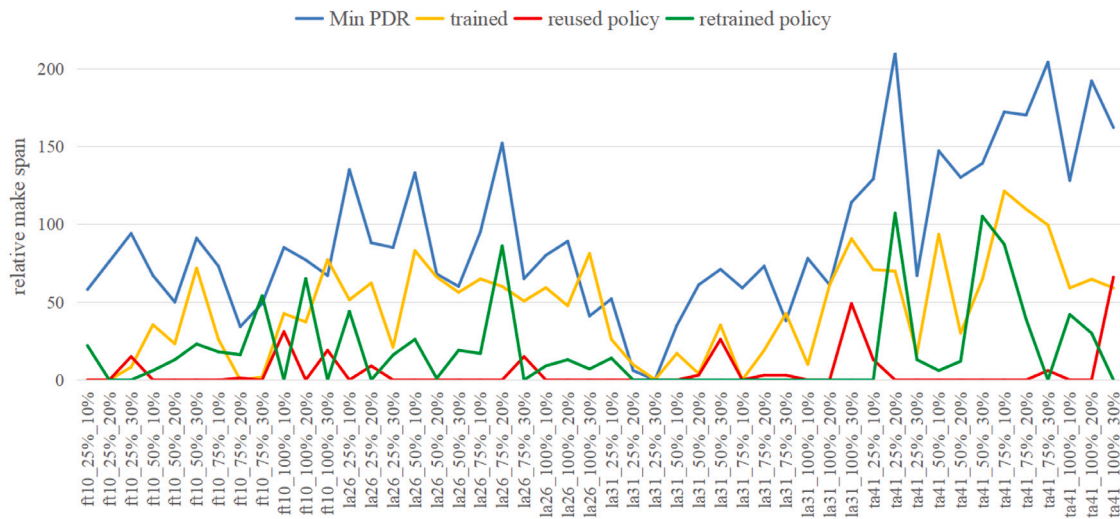


Fig. 11. The make span of four kinds of rescheduling policies on instances with Uniform distribution.

CRedit authorship contribution statement

Xinquan Wu: Conceptualization, Writing – original draft (lead), Review & editing. **Xuefeng Yan:** Supervision, Writing – review & editing. **Donghai Guan:** Review & editing. **Mingqiang Wei:** Review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that support the findings of this study are openly available at [github](https://github.com).

Acknowledgment

This work is supported by the National Defense Basic Scientific Research Program, China under Grant Nos.JCKY2020605C003.

References

- Adams, J., Balas, E., Zawack, D., 1988. The shifting bottleneck procedure for job shop scheduling. *Manage. Sci.* 34 (3), 391–401.
- Asadzadeh, L., 2016. A parallel artificial bee colony algorithm for the job shop scheduling problem with a dynamic migration strategy. *Comput. Ind. Eng.* 102 (dec.), 359–367.
- Aydin, M.E., Öztelmeç, E., 2000. Dynamic job-shop scheduling using reinforcement learning agents. *Robot. Auton. Syst.* 33 (2–3), 169–178.
- Bahdanau, D., Cho, K., Bengio, Y., 2014. Neural machine translation by jointly learning to align and translate. *Comput. Sci.*
- Baykasoglu, Adil, Karaslan, Fatma S., 2017. Solving comprehensive dynamic job shop scheduling problem by using a GRASP-based approach. *Int. J. Prod. Res.* 55 (11–12), 3308–3325.
- Bengio, Yoshua, Lodi, Andrea, Prouvost, Antoine, 2021. Machine learning for combinatorial optimization: A methodological tour d'horizon. *European J. Oper. Res.* 290 (2), 405–421.
- Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M., 2016. Automated design of production scheduling heuristics: A review. *IEEE Trans. Evol. Comput.* 20 (1), 110–124.
- Chen, Ruiqi, Li, Wenxin, Yang, Hongbing, 2022. A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job shop scheduling problem. *IEEE Trans. Ind. Inform.*
- Elsayed, Eman K., Elsayed, Asmaa K., Eldahshan, Kamal A., 2022. Deep reinforcement learning-based job shop scheduling of smart manufacturing. *Comput. Mater. Contin.* 73 (3), 5103–5120.

- Fisher, H., 1963. Probabilistic learning combinations of local job-shop scheduling rules. *Ind. Sched.*
- Garey, M.R., Johnson, D.S., Sethi, Ravi, 1976. The complexity of flowshop and jobshop scheduling. *Math. Oper. Res.* 1 (2), 117–129.
- Gawlikowski, J., Tassi, Crn, Ali, M., Lee, J., Zhu, X.X., 2021. A survey of uncertainty in deep neural networks. *arXiv*.
- Geiger, Christopher D., Kempf, Karl G., Uzsoy, Reha, 1997. A tabu search approach to scheduling an automated wet Etch station. *J. Manuf. Syst.* 16 (2), 102–116.
- Graham, R.L., Lawler, E.L., Lenstra, J.K., Kan, Ahgr, 1979. Optimization and approximation in deterministic sequencing and scheduling: A survey. *Ann. Discrete Math.* 5 (1), 287–326.
- Han, Bao-An, Yang, Jian-Jun, 2020. Research on adaptive job shop scheduling problems based on dueling double DQN. *IEEE Access* 8, 186474–186495.
- Huang, K.L., Liao, C.J., 2008. Ant colony optimization combined with taboo search for the job shop scheduling problem. *Elsevier Sci. Ltd.* 35 (4), 1030–1046.
- Huang, Shengyi, Ontanon, Santiago, 2022. A closer look at invalid action masking in policy gradient algorithms. In: *Proceedings of the International Florida Artificial Intelligence Research Society Conference*, Vol. 35. FLAIRS, Jensen Beach, FL, United states.
- Kayhan, Behice Meltem, Yildiz, Gokalp, 2021. Reinforcement learning applications to machine scheduling problems: a comprehensive literature review. *J. Intell. Manuf.*
- Kim, Youmin, Park, Jinbae, Jang, YounHo, Ali, Muhammad, Oh, Tae-Hyun, Bae, Sung-Ho, 2021. Distilling global and local logits with densely connected relations. In: *2021 IEEE/CVF International Conference on Computer Vision. ICCV*, pp. 6270–6280.
- Lawrence, S., 1984. Supplement to Resource Constrained Project Scheduling: An Experimental Investigation of Heuristic Scheduling Techniques. Graduate School of Industrial Administration.
- Lei, Kun, Guo, Peng, Zhao, Wenchao, Wang, Yi, Qian, Linmao, Meng, Xiangyin, Tang, Liansheng, 2022. A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. *Expert Syst. Appl.* 205.
- Li, Yufeng, He, Yan, Wang, Yulin, Tao, Fei, Sutherland, John W., 2020. An optimization method for energy-conscious production in flexible machining job shops with dynamic job arrivals and machine breakdowns. *J. Clean. Prod.* 254.
- Lin, A., Tsung Lih, C., E. Shi Jinn Horng A B, Kao, D., Tzong Wann, Chen, C. Yuan Hsin, Run, C. Ray Shine, Chen, C. Rong Jian, Lai, C. Jui Lin, Kuo, A. I Hong, 2010. An efficient job-shop scheduling algorithm based on particle swarm optimization. *Expert Syst. Appl.* 37 (3), 2629–2636.
- Liu, Chien-Liang, Chang, Chuan-Chin, Tseng, Chun-Jan, 2020. Actor-critic deep reinforcement learning for solving job shop scheduling problems. *IEEE Access* 8, 71752–71762.
- Liu, Shi Qiang, Kozan, Erhan, 2017. A hybrid metaheuristic algorithm to optimise a real-world robotic cell. *Comput. Oper. Res.* 84, 188–194.
- Luo, Shu, 2020. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Appl. Soft Comput.* 91.
- Luo, Bin, Wang, Sibao, Yang, Bo, Yi, Lili, 2021. An improved deep reinforcement learning approach for the dynamic job shop scheduling problem with random job arrivals. *J. Phys. Conf. Ser.* 1848.
- Manne, A.S., 1960. On the job-shop scheduling problem. *Oper. Res.* 8 (2), 219–223.
- Mattfeld, D.C., Bierwirth, C., 2004. An efficient genetic algorithm for job shop scheduling with tardiness objectives. *European J. Oper. Res.* 155 (3), 616–630.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M., 2013. Playing Atari with deep reinforcement learning. *Comput. Sci.*

- Mohammadi, S., Al-e Hashem, S.M.J., Mirzapour, Rekik, Y., 2020. An integrated production scheduling and delivery route planning with multi-purpose machines: A case study from a furniture manufacturing company. *Int. J. Prod. Econ.* 219, 347–359.
- Monaci, Marta, Agasucci, Valerio, Grani, Giorgio, 2021. An actor-critic algorithm with deep double recurrent agents to solve the job shop scheduling problem. *arXiv*.
- Morinaga, Eiji, Tang, Xuetaian, Iwamura, Koji, Hirabayashi, Naoki, 2023. An improved method of job shop scheduling using machine learning and mathematical optimization. *Procedia Comput. Sci.* 217, 1479–1486, 4th International Conference on Industry 4.0 and Smart Manufacturing.
- Nguyen, S., Mei, Y., Zhang, M., 2017. Genetic programming for production scheduling: a survey with a unified framework. *Complex Intell. Syst.*
- Ouyang, Wenbin, Wang, Yisen, Han, Shaochen, Jin, Zhejiang, Weng, Paul, 2021. Improving generalization of deep reinforcement learning-based tsp solvers. *reinforcement learning*.
- Park, Junyoung, Bakhtiyar, Sanjar, Park, Jinkyoo, 2021a. ScheduleNet: Learn to solve multi-agent scheduling problems with reinforcement learning. *reinforcement learning; combination optimization problem*.
- Park, Junyoung, Chun, Jaehyeong, Kim, Sang Hun, Kim, Youngkook, Park, Jinkyoo, 2021b. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *Int. J. Prod. Res.* 59 (11), 3360–3377, Formulated problems; Generalization capability; Graph neural networks; Graph representation; Policy optimization; Scheduling policies; Scheduling process; Sequential decision making.
- Park, In-Beom, Huh, Jaeseok, Kim, et al., 2020. A reinforcement learning approach to robust scheduling of semiconductor manufacturing facilities. *IEEE Trans. Autom. Sci. Eng.* 17 (3), 1420–1431.
- Qu, Shuhui, Jie, Wang, Shivani, Govil, 2016. Learning adaptive dispatching rules for a manufacturing process system by using reinforcement learning approach. In: 2016 IEEE 21st International Conference on Emerging Technologies and Factory Automation, Vol. 2016-November. ETFA, Berlin, Germany, Adaptiveness; Approximate methods; Dispatching rules; Job shop; Multi stage; Multi-machines; Multi-products; Setup costs; Stochastic processing time.
- Roy, Arunabha M., Bhaduri, Jayabrata, 2022. Real-time growth stage detection model for high degree of occlusion using DenseNet-fused YOLOv4. *Comput. Electron. Agric.* 193, 106694.
- Roy, Arunabha M., Bose, Rikhi, Bhaduri, Jayabrata, 2022. A fast accurate fine-grain object detection model based on YOLOv4 deep neural network. *Neural Comput. Appl.* 34 (5), 3895–3921.
- Russo, Luigi, Terlizzi, Mario, Tipaldi, Massimo, Glielmo, Luigi, 2021. A reinforcement learning approach for pedestrian collision avoidance and trajectory tracking in autonomous driving systems. In: 2021 5th International Conference on Control and Fault-Tolerant Systems. SysTol, pp. 44–49.
- Schaul, Tom, Quan, John, Antonoglou, Ioannis, Silver, David, 2016. Prioritized experience replay. In: 4th International Conference on Learning Representations, ICLR 2016 - Conference Track Proceedings. San Juan, Puerto rico, Human-level performance; Reinforcement learning agent; State of the art.
- Schulman, John, Wolski, Filip, Dhariwal, Prafulla, Radford, Alec, Klimov, Oleg, 2017. Proximal policy optimization algorithms. *arXiv*.
- Sels, Veronique, Gheysen, Nele, Vanhoucke, Mario, 2012. A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. *Int. J. Prod. Res.* 50 (15), 4255–4270.
- Shady, Salama, Kaihara, Toshiya, Fujii, Nobutada, Kokuryo, Daisuke, 2021. A new representation and adaptive feature selection for evolving compact dispatching rules for dynamic job shop scheduling with genetic programming. In: *Advances in Production Management Systems. Artificial Intelligence for Sustainable and Resilient Production Systems*, Vol 632 IFIP. Nantes, France, pp. 646–654, Adaptive feature selection; Computational power; Dispatching rules; Dynamic job shop scheduling; Features selection; Industrial practices; Job-Shop scheduling; Manual design; Search spaces; Selection mechanism.
- Shady, Salama, Kaihara, Toshiya, Fujii, Nobutada, Kokuryo, Daisuke, 2022. A novel feature selection for evolving compact dispatching rules using genetic programming for dynamic job shop scheduling. *Int. J. Prod. Res.* 60 (13), 4025–4048, Automated design; Computational power; Discrete-event simulations; Dispatching rules; Dynamic job shop scheduling; Features selection; Job shop scheduling problems; Machine learning algorithms; Scheduling rules; Search spaces.
- Shaheen, Basheer, Kocsis, Ádám, Németh, István, 2023. Data-driven failure prediction and RUL estimation of mechanical components using accumulative artificial neural networks. *Eng. Appl. Artif. Intell.* 119, 105749.
- Shaheen, Basheer Wasef, Németh, István, 2022. Integration of maintenance management system functions with industry 4.0 technologies and features a review. *Processes* 10 (11).
- Shahrabi, J., Adibi, M.A., Mahootchi, M., 2017. A reinforcement learning approach to parameter estimation in dynamic job shop scheduling. *Comput. Ind. Eng.* 110 (aug.), 75–82.
- Shahzad, A., Mebarki, N., 2012. Data mining based job dispatching using hybrid simulation-optimization approach for shop scheduling problem. *Eng. Appl. Artif. Intell.* 25 (6), 1173–1181.
- Song, Wen, Chen, Xinyang, Li, Qiqiang, Cao, Zhiguang, 2022. Flexible job shop scheduling via graph neural network and deep reinforcement learning. *IEEE Trans. Ind. Inform.* 1–11.
- Tai, L., Paolo, G., Liu, M., 2017. Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation. In: *Intelligent Robots and Systems (IROS)*, 2017 IEEE/RSJ International Conference on.
- Taillard, E.D., 1993. Benchmarks for basic scheduling problems. *European J. Oper. Res.* 64 (2), 278–285.
- Tassel, Pierre, Gebser, Martin, Schekotihin, Konstantin, 2021. A reinforcement learning environment for job-shop scheduling. *Automated systems; Combinatorial optimization problems; Job-shop; Learning environments; Optimal schedule; Production cost; Production wastes; State representation; System applications*.
- Turgut, Yakup, Bozdog, Cafer Erhan, 2020. Deep Q-network model for dynamic job shop scheduling pproblem based on discrete event simulation. In: 2020 Winter Simulation Conference, Vol. 2020-December. WSC, Orlando, FL, United states, pp. 1551–1559.
- Ullah, Irfan, Khan, Sharifullah, Imran, Muhammad, Lee, Young-Koo, 2021. RweetMiner: Automatic identification and categorization of help requests on twitter during disasters. *Expert Syst. Appl.* 176, 114787.
- Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, Polosukhin, Illia, 2017. Attention is all you need. *Adv. Neural Inf. Process. Syst.* 2017-December, 5999–6009.
- Wang, Libing, Hu, Xin, Wang, Yin, Xu, Sujie, Ma, Shijun, Yang, Kexin, Liu, Zhijun, Wang, Weidong, 2021. Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning. *Comput. Netw.* 190.
- Wang, Xu, Wang, Sen, Liang, Xingxing, Zhao, Dawei, Huang, Jincai, Xu, Xin, Dai, Bin, Miao, Qiguang, 2022. Deep reinforcement learning: A survey. *IEEE Trans. Neural Netw. Learn. Syst.* 1–15.
- Yahyaoui, Amel, Fnaiech, Nader, Fnaiech, Farhat, 2011. A suitable initialization procedure for speeding a neural network job-shop scheduling. *IEEE Trans. Ind. Electron.* 58 (3), 1052–1060.
- Yang, Shanggan, 2022. Using attention mechanism to solve job shop scheduling problem. In: 2022 2nd International Conference on Consumer Electronics and Computer Engineering. ICCECE 2022, Guangzhou, China, pp. 59–62, Attention; Attention mechanisms; Disjunctive graphs; Feature information; Job shop scheduling problems; Job-shop scheduling problem; Learn-; Policy networks; Reduce time; Structure information.
- Yim, Seong Jin, Lee, Doo Yong, 1999. Scheduling cluster tools in wafer fabrication using candidate list and simulated annealing. *J. Intell. Manuf.* 10 (6), 531–540, Scheduling cluster tools.
- Yu, James J.Q., Yu, Wen, Gu, Jiatao, 2019. Online vehicle routing with neural combinatorial optimization and deep reinforcement learning. *IEEE Trans. Intell. Transp. Syst.* 20 (10), 3806–3817.
- Yun, S., Choi, J., Yoo, Y., Yun, K., Jin, Y.C., 2017. Action-decision networks for visual tracking with deep reinforcement learning. In: *IEEE Conference on Computer Vision and Pattern Recognition*.
- Zeng, Zhengqi, Li, Xiaoxia, Bai, Changbo, 2022a. A deep reinforcement learning approach to flexible job shop scheduling. In: *Conference Proceedings - IEEE International Conference on Systems, Man and Cybernetics*, Vol. 2022-October. Prague, Czech republic, pp. 884–890, Actor-critic algorithm; Deep reinforcement learning; Disjunctive graphs; Flexible job-shop scheduling; Graph isomorphic network; Job-Shop scheduling; Machining process optimization; Reinforcement learning approach; Reinforcement learnings; Scheduling rules.
- Zeng, Yunhui, Liao, Zijun, Dai, Yuanzhi, Wang, Rong, Li, Xiu, Yuan, Bo, 2022b. Hybrid intelligence for dynamic job-shop scheduling with deep reinforcement learning and attention mechanism. *arXiv*.
- Zhang, Fangfang, Mei, Yi, Nguyen, Su, Zhang, Mengjie, 2021. Evolving scheduling heuristics via genetic programming with feature selection in dynamic flexible job-shop scheduling. *IEEE Trans. Cybern.* 51 (4), 1797–1811, Adaptation strategies; Combinatorial optimization; Complementary features; Dynamic environments; Flexible job-shop scheduling; Performance limitations; Redundant features; Scheduling heuristics.
- Zhang, Cong, Song, Wen, Cao, et al., 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning. *arXiv*.
- Zhao, Linlin, Shen, Weiming, Zhang, Chunjiang, Peng, Kunkun, 2022. An end-to-end deep reinforcement learning approach for job shop scheduling. In: 2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design. CSCWD 2022, Hangzhou, China, pp. 841–846.
- Zhao, Yeyian, Wang, Yanhong, Tan, Yuanyuan, Zhang, Jun, Yu, Hongxia, 2021. Dynamic jobshop scheduling algorithm based on deep Q network. *IEEE Access* 9, 122995–123011.
- Zheng, Guanjie, Zhang, Fuzheng, Zheng, Zihan, Xiang, Yang, Yuan, Nicholas Jing, Xie, Xing, Li, Zhenhui, 2018. DRN: A deep reinforcement learning framework for news recommendation. In: *Proceedings of the 2018 World Wide Web Conference. WWW '18, International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE*, pp. 167–176.
- Zhou, J., Li, P., Zhou, Y., Wang, B., Meng, L., 2018. Toward new-generation intelligent manufacturing. *Engineering* 4 (4), 11–20.
- Zhou, Longfei, Zhang, Lin, Horn, Berthold K.P., 2020. Deep reinforcement learning-based dynamic scheduling in smart manufacturing. *Procedia CIRP* 93, 383–388, 53rd CIRP Conference on Manufacturing Systems 2020.